Week 10 RTL Lab: Debugging a Broken Snooping Invalidation Interface

1. Exercise Goal

1. Understand how individually correct components can fail when the top-level interface is wired incorrectly.
2. Debug a two-core, two-private-cache, shared-memory prototype using Verilator traces and printed debug signals.
3. Fix a simple snooping invalidation interface, not a full MSI protocol FSM.
4. Finish the lab in approximately 1.5 hours during class.

2. Architecture Explanation

System Overview

- Two processor cores connect to private one-line write-through caches and a shared main memory.
- Both caches may hold copies of the same memory block, creating shared-state correctness risks.
- Snooped invalidation removes stale peer copies after writes.
- The exercise studies how this interface preserves correctness in a shared-memory prototype.

Stale-Read Failure Example

- Address 0x24 begins with value 10.
- Both cores read and cache the value.
- One core writes a new value while the other may keep a stale copy if invalidation fails.
- The exercise begins from this failure and asks students to debug the cause.

Invalidate Interface Contract

- A write must generate an invalidate pulse.
- The full address must be delivered to the peer cache.
- The peer cache must receive and act on the invalidation.
- Breaking any step violates shared-memory correctness.

Signals and Observability

- Use trace outputs to observe valid bits, tags, data, invalidate signals, snoop signals, and memory values.
- Compare cache contents against memory to detect stale data.
- Differentiate between invalidate generation and invalidate reception.

Task Boundary

- Students modify only tiny_system_todo.sv.
- Cache, memory, and testbench infrastructure are assumed correct.
- The goal is integration debugging, not implementing a full coherence protocol.

Discussion Prompt

- If both caches are individually correct, how can the integrated multicore system still fail?

3. Docker Setup Commands

5. Pull the image: docker pull amansinhaatnycu/osp:latest
6. Enter the container from the directory containing the unzipped lab folder: docker run --rm -it -v "$PWD":/workspace amansinhaatnycu/osp:latest bash
7. Go to the base lab: cd /workspace/osp_w10_snoop_interface_lab_FINAL/base
8. Check the files in rtl, tb, scripts and docs

4. Compile and Run the Starter

9. Make the script executable if needed: chmod +x scripts/run_verilator.sh
10. Compile and run: ./scripts/run_verilator.sh
11. The script runs Verilator, builds the C++ testbench, and executes build/obj_dir/simv automatically.
12. The starter should compile and run, but the simulation should fail because the top-level invalidation interface is intentionally broken.

5. Expected Starter Behavior

13. The first reads should observe the initial memory value 10 at address 0x24.
14. After Core 0 writes 42, Core 1 should reread 42, but the broken starter may keep stale data if the invalidation address is wrong.
15. After Core 1 writes 99, Core 0 should reread 99, but the broken starter may keep stale data if the invalidation pulse or address is wrong.
16. Students should use the printed debug columns to identify whether the invalidate pulse and address reached the peer cache.

6. What Students Are Allowed To Edit

17. Edit only: rtl/tiny_system_todo.sv
18. Do not edit: rtl/tiny_cache.sv
19. Do not edit: tb/tb_tiny_system.cpp
20. Do not edit: scripts/run_verilator.sh
21. Do not solve the problem by changing the testbench expectations.

7. The Three Intended Bugs

22. Bug 1: Core 1 write invalidation toward Core 0 is suppressed.
23. Bug 2: Core 1 invalidation address toward Core 0 is truncated.
24. Bug 3: Core 0 invalidation address toward Core 1 is truncated.

25. The correct design sends the invalidate pulse from the writer cache to the peer cache and preserves the full address.

8. Debugging Instructions

26. Run the starter once and read the first failing line.
27. Look at the trace columns for inv0, inv1, snoop0, and snoop1.
28. Check whether a write produces an invalidate request from the writing core.
29. Check whether the peer receives a snoop valid pulse.
30. Check whether the peer receives the full address 0x24 or the incorrect truncated address 0x04.
31. Fix one assignment at a time, then rerun ./scripts/run_verilator.sh.

9. Expected Final Behavior

32. The final simulation should print: [PASS] Snooping invalidation interface is correctly integrated.
33. Core 1 reread after Core 0 write should observe 42.
34. Core 0 reread after Core 1 write should observe 99.
35. Memory at address 0x24 should end with value 99.

---

10. Correct Fixes

36. assign snoop_to_c0_valid = c1_inv_valid;
37. assign snoop_to_c0_addr = c1_inv_addr;
38. assign snoop_to_c1_valid = c0_inv_valid;
39. assign snoop_to_c1_addr = c0_inv_addr;
40. These four assignments implement the intended two-cache snooping invalidation interface.

11. Discussion Points

41. Why did correct cache modules fail when integrated incorrectly?
42. Which bug was easier to find: missing pulse or wrong address?
43. Why is full-width address propagation an interface contract?
44. How would this exercise become harder with four or eight cores?
45. How is this different from implementing a full MSI/MESI protocol?
46. Why are debug signals and deterministic tests important for open-source prototype systems?
47. How does this lab connect Week 9 system integration to Week 10 coherence?